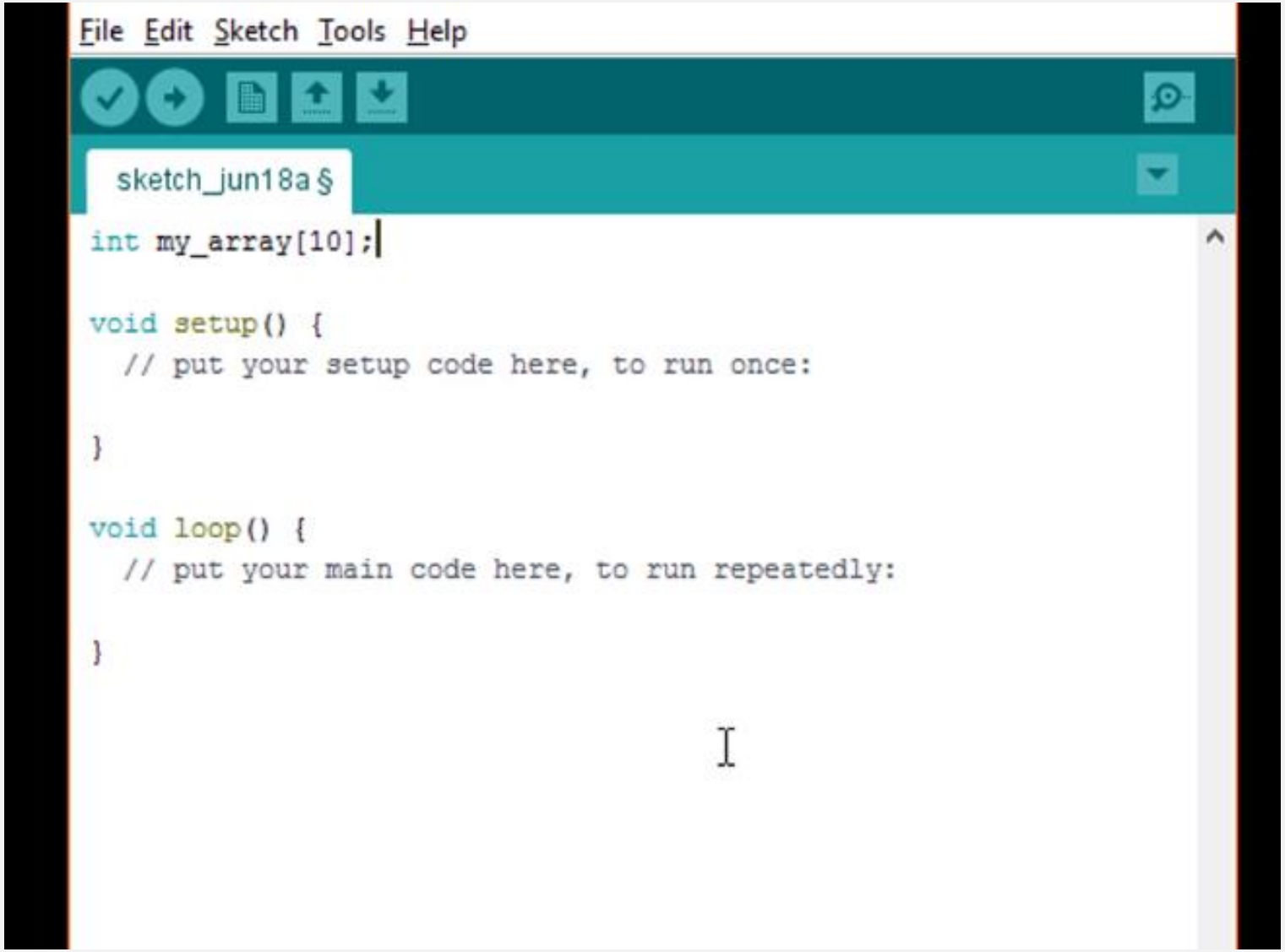
بسم الله الرحمن الرحيم

آرایه ها و رشته ها

فرض کنید ۵۰ مقدار از یک نوع دارید که میخواهید در یک برنامه از آن استفاده کنید. مثلاً ۵۰ چراغ داریم که میخواهیم یادمان باشد کدام ها روشن و کدام خاموش هستند.
یک روش این است که ۵۰ متغیر جدا از هم تعریف کنیم ولی این کار بهینه نمیباشد و قابلیت خوانایی آن را کم می کند.
روش دیگر که بهینه است استفاده از آرایه می باشد:

```
Int my_array[10];
```


File Edit Sketch Tools Help
sketch_jun18a §
int my_array[10];
void setup() {
// put your setup code here, to run once:
}
void loop() {
// put your main code here, to run repeatedly:
}

آرایه ها و رشته ها

می بینیم که یک آرایه گلوبال به نام my_array تعریف کردیم.

نکته: تمامی المان های درون یک آرایه باید از یک نوع یکسان باشد که در مثال ما نوعشان int می باشد. این مورد مانند اعلان یک متغیر تک می باشد با این تفاوت که در پایان متغیر براکت قرار می دهیم و بین براکت یک عدد داریم که نشان دهنده ی چه تعداد المان این آرایه دارد.

در اینجا ما آرایه را اعلان کردیم بنابراین می بایست درون هر المان یک متغیر قرار دهیم. برای این کار استفاده از حلقه های for به ما کمک می کنند.

```
int my_array[10];

void setup() {
  Serial.begin(9600);

  for ( int i = 0; i < 10; i++ ) {
    my_array[i] = i * 5;
  }
}

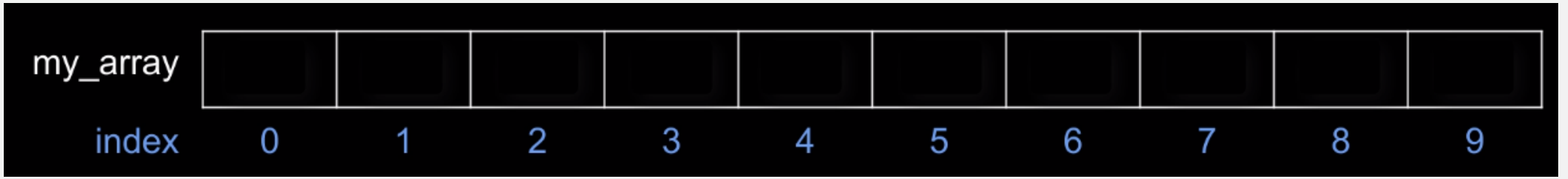
void loop() {
  // put your main code here, to run repeatedly:

}
```

کاری که در اینجا میکنیم این است که مضارب ۵ را میشماریم و درون آرایه ذخیره می کنیم.

آرایه ها و رشته ها

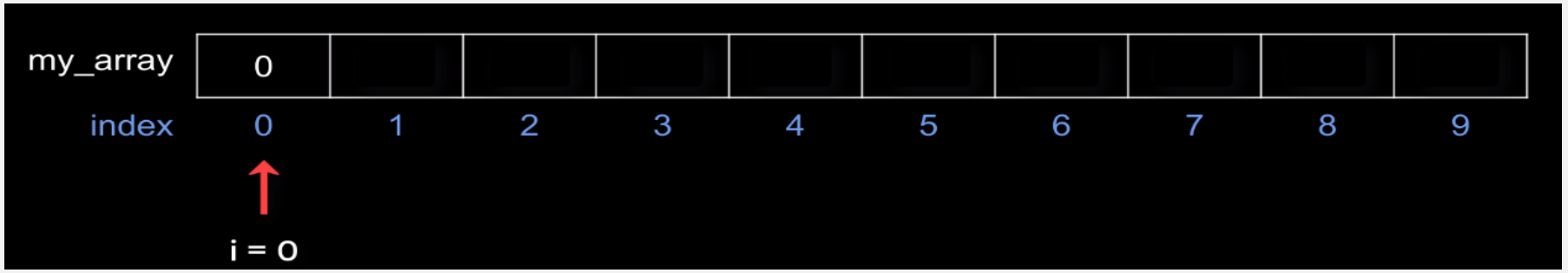
با یک تصویر نشان می دهیم چه اتفاقی میوفتد.
وقتی که برنامه را اجرا می کنیم یک آرایه ایجاد کردیم که ۱۰ المان دارد. این المان ها با اعداد ۰ تا ۹ قابل دسترسی اند.



با اینکه آرایه را اعلان کردیم عددی را برای المان های آن در نظر نگرفتیم و هر المان یک int می باشد.
بنابراین در تابع setup یک حلقه for ایجاد کردیم که هر المان را درنظر گرفته و یک مقدار به آن تخصیص می دهد.

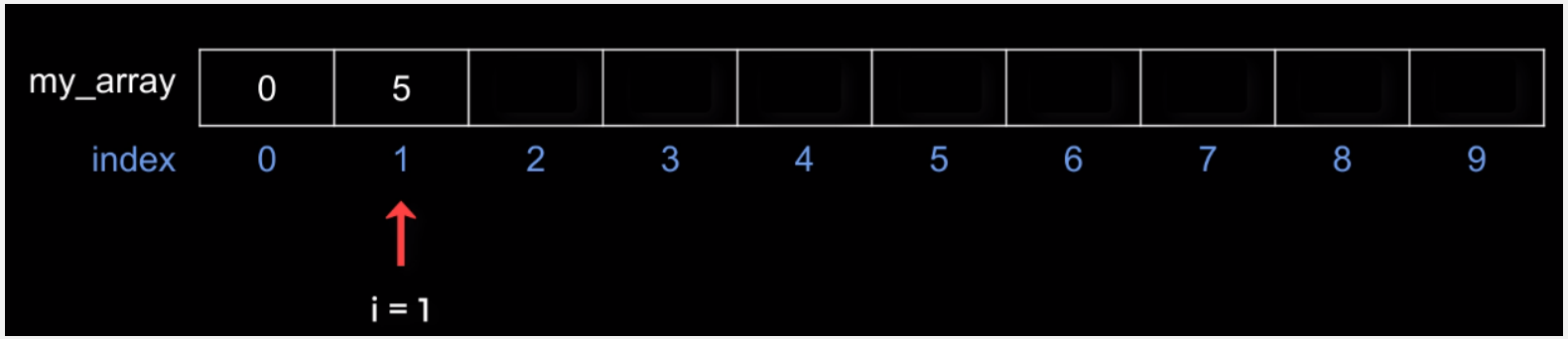
آرایه ها و رشته ها

تکرار اول حلقه:

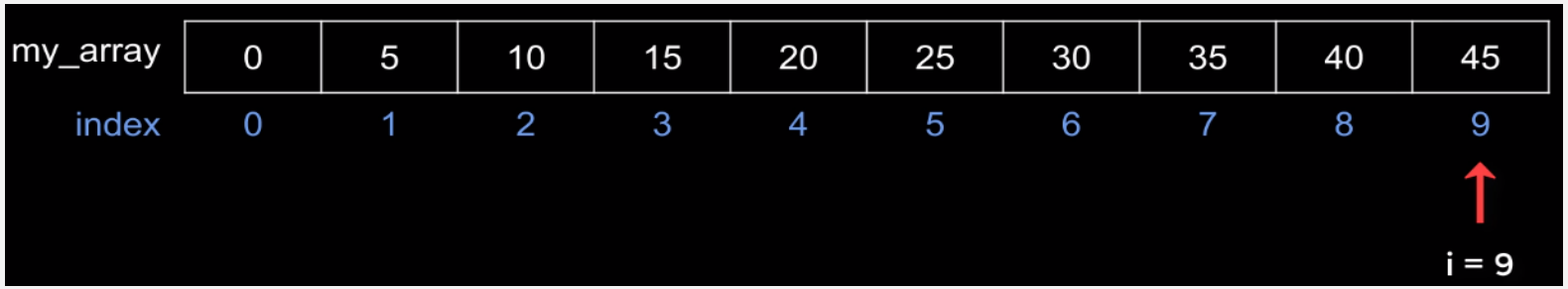


تکرار دوم حلقه:

.
.
.



تکرار ۹(آخر) حلقه:



آرایه ها و رشته ها

به این صورت به آرایه ها دسترسی پیدا می کنیم و مقادیر برای المان های آن تخصیص می دهیم.

حال چگونه این مقادیر را از المان ها می خوانیم؟
دقیقا به صورت قبل

```
int my_array[10];

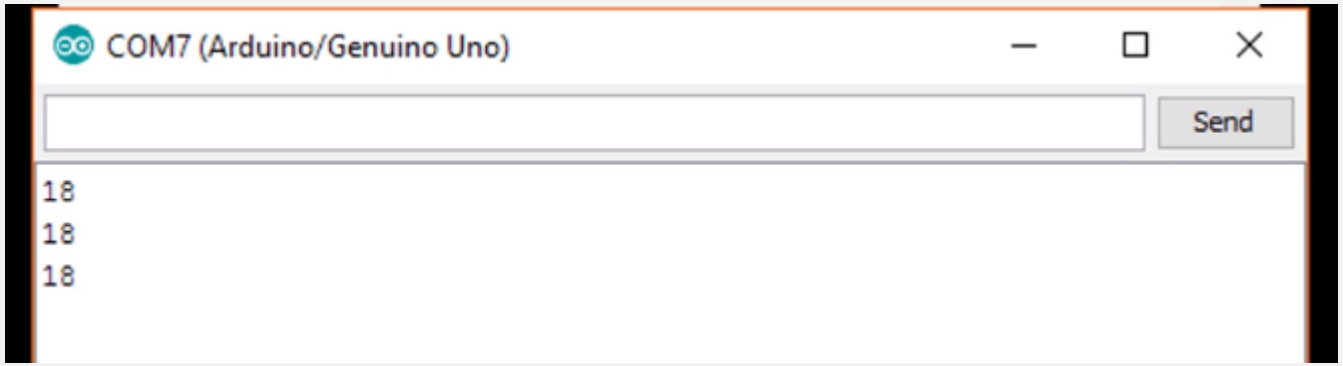
void setup() {
  Serial.begin(9600);

  for ( int i = 0; i < 10; i++ ) {
    my_array[i] = i * 5;
  }
}

void loop() {
  Serial.println(my_array[3] + 3);
  delay(1000);
}
```

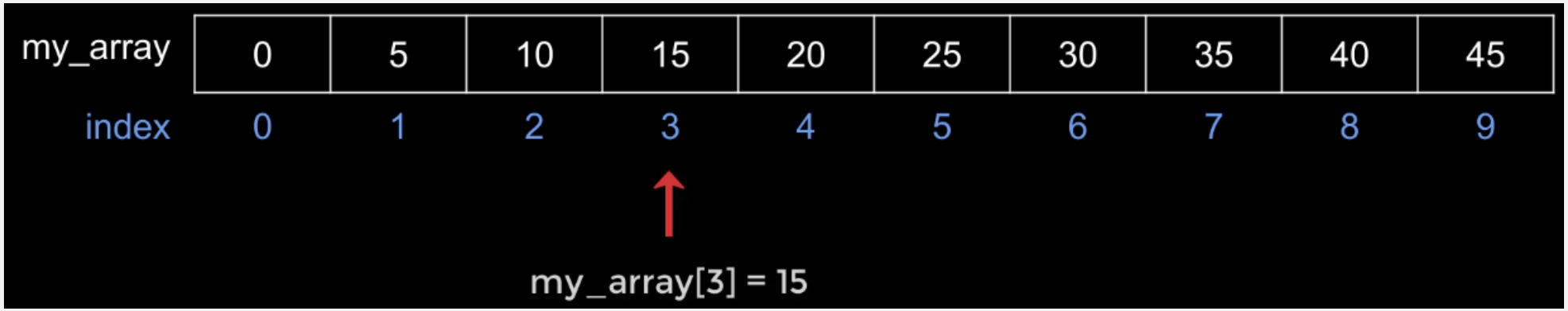
آرایه ها و رشته ها

در خروجی عدد ۱۸ را خواهیم دیم که هر ۱ ثانیه پرینت می شود:



آرایه ها و رشته ها

با ملاحظه به دیاگرام داریم:



15 + 3 = 18

اگر می خواستیم تمامی مقادیر درون آرایه را چاپ کنیم؟

آرایه ها و رشته ها

```
int my_array[10];

void setup() {
  Serial.begin(9600);

  for ( int i = 0; i < 10; i++ ) {
    my_array[i] = i * 5;
  }
}

void loop() {
  for ( int i = 0; i < 10; i++ ) {
    Serial.print(my_array[i]);
    Serial.print(" ");
  }
  Serial.println();
  delay(1000);
}
```



آرایه ها و رشته ها

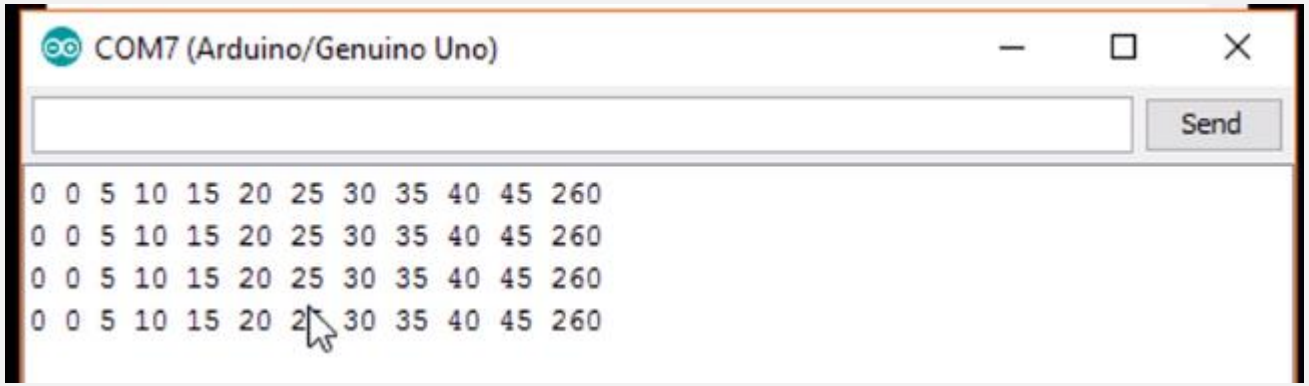
خروجی؟

```
int my_array[10];

void setup() {
  Serial.begin(9600);

  for ( int i = 0; i < 10; i++ ) {
    my_array[i] = i * 5;
  }
}

void loop() {
  for ( int i = -1; i < 11; i++ ) {
    Serial.print(my_array[i]);
    Serial.print(" ");
  }
  Serial.println();
  delay(1000);
}
```



در زبان سی برای فراخوانی خارج از محدوده خطایی نمایش داده نمیشود. در خروجی برای المان ۱- و ۱۰ یک مقدار رندوم درج می شود. این به این خاطر است که در زبان سی آرایه ها در حافظه همراه با سایر متغیرها و مقادیر برنامه ذخیره می شوند.

وقتی که یک مقدار اندیس خارج از محدوده آرایه ایجاد می گردد، هر چیزی که در آن قسمت برنامه وجود داشته باشد برای ما نمایش داده میشود. و این مقدار هرچیزی می تواند باشد چراکه ما نمیدانیم چگونه کامپایلر حافظه را تنظیم و مرتب می کند.

بنابراین این نکته خیلی مهم هست چرا که میتوانیم مقدار دیگری را overwrite کنیم ویا برنامه را crash کنیم (با overwrite کردن قسمتی از حافظه برنامه).

برای جلوگیری از این کار همیشه چک کنید که چگونه آرایه ورودیتان را اندیس گزاری می کند.

آرایه ها و رشته ها

برای این مورد می توان از constant ها استفاده کرد به جای اینکه هر بار از اعداد استفاده کنیم. با این کار به کامپایلر میگیم که این متغیر هیچگاه در طی برنامه تغییر نمیکند.

```
const int len = 10;
int my_array[len];

void setup() {
  Serial.begin(9600);

  for ( int i = 0; i < len; i++ ) {
    my_array[i] = i * 5;
  }
}

void loop() {
  for ( int i = 0; i < len; i++ ) {
    Serial.print(my_array[i]);
    Serial.print(" ");
  }
  Serial.println();
  delay(1000);
}
```



حال با اینکار اگر نیاز داشته باشید طول آرایه را در برنامه تغییر دهید تنها نیاز دارید یک عدد را تغییر دهید.

آرایه ها و رشته ها

روشی دیگر برای مقدار دهی اولیه به آرایه:

```
const int len = 10;
int my_array[len] = {0, 5, 10, 15, 20};

void setup() {
  Serial.begin(9600);
}

void loop() {
  for ( int i = 0; i < len; i++ ) {
    Serial.print(my_array[i]);
    Serial.print(" ");
  }
  Serial.println();
  delay(1000);
}
```

در روبرو یک آرایه با ۱۰ المان تعریف کرده ایم ولی مقادیر به ۵ المان اولی آرایه تخصیص داده ایم.

آرایه ها و رشته ها

اگر بخواهیم تنها ۵ المان اول مقدار تخصیص بگیرند:

```
const int len = 10;
int my_array[len] = {0, 5, 10, 15, 20};

void setup() {
  Serial.begin(9600);
}

void loop() {
  for ( int i = 0; i < len; i++ ) {
    Serial.print(my_array[i]);
    Serial.print(" ");
  }
  Serial.println();
  delay(1000);
}
```

```
0 5 10 15 20 0 0 0 0 0
0 5 10 15 20 0 0 0 0 0
```

در خروجی می بینیم که المان هایی که مقدار تخصیص داده نشده اند با صفر پر میشوند.
ولی باید در نظر بگیرید که المان هایی که مقدار به آن تخصیص داده نمیشود هر عدد رندومی می تواند باشد.

آرایه ها و رشته ها

اگر بخواهیم بیش از مقادیر المان آرایه تخصیص دهیم، با خطای کامپایلر مواجه خواهیم شد:

```
const int len = 10;
int my_array[len] = {0, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50};

void setup() {
  Serial.begin(9600);
}

void loop() {
  for ( int i = 0; i < len; i++ ) {
    Serial.print(my_array[i]);
    Serial.print(" ");
  }
  Serial.println();
  delay(1000);
}
```

too many initializers for 'int [10]'

Copy error messages

```
exit status 1
too many initializers for 'int [10]'
```

آرایه ها و رشته ها

حتی می توانید به صورت روبرو آرایه را مقدار دهی کنید:

```
const int len = 10;
int my_array[] = {0, 5, 10, 15, 20};

void setup() {
  Serial.begin(9600);
}

void loop() {
  for ( int i = 0; i < len; i++ ) {
    Serial.print(my_array[i]);
    Serial.print(" ");
  }
  Serial.println();
  delay(1000);
}
```

خروجی: ؟

آرایه ها و رشته ها

از آنجایی که متغیر len را آپدیت نکردیم. با اجرای قطعه کد قبل سعی میکنیم المان هایی که از آخر آرایه گذشته اند را پرینت کنیم.

درنتیجه مقادیری که تخصیص داده بودیم را به همراه ۵ عدد رندوم دیگر در خروجی خواهیم داشت.

```
0 5 10 15 20 32 2573 0 0 0
0 5 10 15 20 32 2573 0 -5829 3
0 5 10 15 20 32 2573 0 -11917 7
0 5 10 15 20 32 2573 0 -17615 11
0 5 10 15 20 32 2573 0 -23444 15
0 5 10 15 20 32 2573 0 -29401 19
0 5 10 15 20 32 2573 0 30306 23
```

بنابراین نیازداریم مقدار متغیر len را از ۱۰ به ۵ تغییر دهیم تا برنامه کار کند:

```
0 5 10 15 20
0 5 10 15 20
0 5 10 15 20
0 5 10 15 20
```

آرایه ها و رشته ها

حال به سراغ نوع خاصی از آرایه ها یعنی string میرویم.

String یک آرایه از کاراکترهاست که میتواند شامل اعداد، حروف، فاصله و هر کاراکتر ویژه دیگری از جدول Ascii باشد.

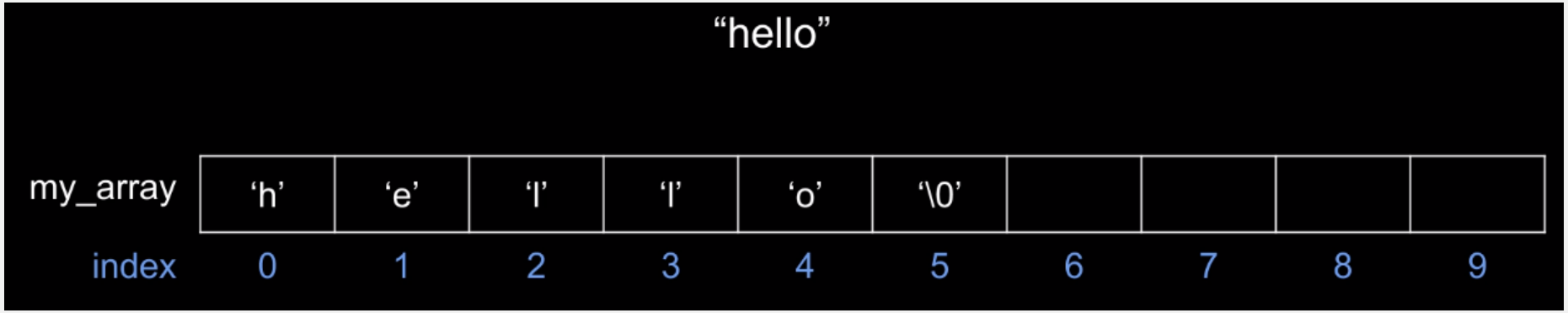
```
const int len = 5;
char my_array[len] = "hello";

void setup() {
  Serial.begin(9600);
}
```

به یاد داشته باشید که هر کاراکتر یک المان می باشد.
در بالا کلمه hello دارای ۵ کاراکتر می باشد. در واقع یک آرایه با حداقل ۶ المان نیاز داریم تا string را ذخیره کند. چرا؟

آرایه ها و رشته ها

زمانی که یک string با "" ایجاد میکنیم، کامپایلر به صورت اتوماتیک یک المان از هر کاراکتر ایجاد کرده و یک کاراکتر null به انتهای رشته میفزاید.



این کاراکتر null، که مقدار ۰ را در جدول Ascii دارد با استفاده از دو کاراکتر تعریف می شود (با اینکه یک کاراکتر در نظر گرفته میشود).

\ is the escape character
\0 is the null character

کاراکتر \ وقتی با برخی سایر کاراکتر ها استفاده می شود نشان دهنده ی کاراکترهای ویژه ای خواهد بود که در اکثر کیبوردها پیدا نمیشود.

در اینجا \0 به کاراکتر null اشاره می کند و به صورت اتوماتیک توسط کامپایلر اضافه می گردد. مابقی چهار المان با مقادیر ناشناخته پر می شوند.

```
const int len = 10;
char my_array[len] = "hello";

void setup() {
  Serial.begin(9600);
}

void loop() {
  for ( int i = 0; i < len; i++ ) {
    Serial.print(my_array[i]);
  }
  Serial.println();
  delay(1000);
}
```



آرایه ها و رشته ها

برنامه قبل به نظر درست مقدار را چاپ می کند ولی ممکن است برنامه سعی کند مقادیری که در string وجود ندارند را چاپ کند.
احتمالاً مقادیر دیگر ۰ باشند ولی به صورت قطع نمیدانیم و ممکن است با مقادیر ناشناخته ای مواجه شویم که پرینت شوند.
برای جلوگیری از این امر می توانیم یک چک کردن اضافه کنیم که زمانی که کاراکتر null را در حلقه for میبینیم

```
const int len = 10;
char my_array[len] = "hello";

void setup() {
  Serial.begin(9600);
}

void loop() {
  for ( int i = 0; i < len; i++ ) {
    if ( my_array[i] == '\0' ) {
      break;
    }
    Serial.print(my_array[i]);
  }
  Serial.println();
  delay(1000);
}
```

با اینکار هر کاراکتر چک می شود و اگر کاراکتر null بود، قبل از چاپ آن از حلقه خارج می شود.

آرایه ها و رشته ها

روشی ساده تر برای پرینت string در ترمینال:

```
const int len = 10;
char my_array[len] = "hello";

void setup() {
  Serial.begin(9600);
}

void loop() {
  Serial.println(my_array);
  delay(1000);
}
```

توابع Print و println به نحوی نوشته شده اند که به راحتی با string کار میکنند و تمامی چک کردن های اضافی را به صورت خود به خود بر روی string انجام می دهند.

نکته: به یاد داشته باشید که یک المان بیشتر از تعداد کاراکتری که در یک آرایه نیاز دارید، در نظر بگیرید.
حتی می توانید طول آرایه را به عهده کامپایلر بگزارید:
به این صورت که چیزی بین براکت ها نمیگزاریم:

```
const int len = 10;
char my_array[] = "hello";

void setup() {
  Serial.begin(9600);
}

void loop() {
  Serial.println(my_array);
  delay(1000);
}
```

به یاد داشته باشید که اینکار زمانی مناسب هست که string عوض نمی شود.

Practice:

Open a new Arduino sketch and copy the following lines to the top (creating 2 global variables):

```
const int len = 10;
int a[len] = {0, 2, -4, 12, -52, 42, -96, 7, -23, 99};
```

In setup(), compute the average of the a[] array and print it to the Serial Monitor.

In this case, the average of a[] is -1.3. Without modifying your code, you should be able to change the global variables to

```
const int len = 8;
int a[len] = {125, 1920, 503, 299, 67, 13578, 7632, 22043};
```

and get an average of 5770.8750.

Hint: You can use Serial.println(float_variable, 4); to print 4 decimal places.